Algorithmen und Datenstrukturen
Übung 6 – Sequentielle Container

Aufgabe 1: Theorie

a) Gegeben sei das folgende Array:

```
5, 1, 10, 19, 23, 18, 10, 4, 10, 4
```

Fügen Sie mit dem Verfahren aus der Vorlesung an Index 4 einen neuen Eintrag in das Array ein. Zeichnen Sie (gemäß VL), welche Kopien laufen (d. h. von welchem Index zu welchem Index kopiert wird).

b) Warum ist ein Array als Grundlage für eine FIFO-Queue nicht gut geeignet?

c) Gegeben sei eine doppelt verkette Liste mit folgenden Daten:

```
17, 8, 6, 2, 2, 11, 6, 5
```

Löschen Sie der Reihe nach die Einträge 17, 5, 8 und 11. Welche Zeiger sind dabei wie anzupassen? Zeichnen Sie die zum Verständnis nötigen Zwischenschritte.

d) Könnte man einen Stack auch auf Grundlage einer Liste aufbauen? Wo liegen die Vorteile, wo liegen die Nachteile?

e) Beantworten Sie folgende Grundlagenfragen:

- Was ist ein impliziter Verweis?
- Warum ist es nicht sinnvoll, die Knoten eines Heaps als Knoten in einer expliziten Baumdatenstruktur zu realisieren?
- Welche Nachteile hat es, wenn man die Verweise eines Array-Heaps ausdrücklich als Zeiger speichert?

f) Gegeben sei folgendes Array.

```
1, 9, 42, 10, 48, 50, 49, 11, 49, 42, 51, 57
```

Handelt es sich um einen Min-Heap? Begründen Sie!

g) Gegeben sei folgender Min-Heap.

```
1, 5, 9, 7, 14, 21, 10, 13, 16, 23, 15, 25, 26, 17, 19
```

Zeichnen Sie den Heap als Schichtenstruktur, als Baum sowie als Array mit impliziten Verweisen.

h) Gegeben ein (kaputter) Heap, der bis auf einen Teilast bereits die Heap-Eigenschaft erfüllt. Gerade ist das Kind c_l durch ein Sift-Up verändert wurden. Warum genügt es, nur mit dem Elternknoten p zu tauschen, um auch die Heap-Eigenschaft bezüglich des anderen Kinds c_r herzustellen?

i) Sortieren Sie das folgende Array absteigend mittels Heap-Sort. Zeichnen Sie alle wesentlichen Zwischenschritte (inklusive allen nötigen Tauschschritten) als Array.

```
5, 4, 1, 9, 6, 2, 10, 11, 3
```

Prüfen Sie hernach mittels binärer Suche aus der Vorlesung, ob das Element 7 in dem Array enthalten ist. Geben Sie erneut alle wichtigen Zwischenschritte an.

j) Beweisen Sie: In einem als Array gefassten Heap errechnet sich aus dem Index i_p des Elternknotens...

- ... der Index $i_l = 2i_p + 1$ des linken Kinds...
- ... und der Index $i_r = 2i_p + 2$ des rechten Kinds.

k) Sorgt es für Probleme, wenn ein Heap die selbe Zahl mehrfach enthält? Wie müsste man die Regeln anpassen, dass der Heap weiterhin funktioniert? Begründen Sie!

Hinweis: Die nötige Anpassung ist denkbar einfach und rein formaler Natur.

l) Beweisen Sie: In einem Heap mit n Elementen gilt für die Höhe h (d. h. für die Zahl der Schichten, die zumindest einen Knoten enthalten), dass $h \in O(\log_2 n)$.

m) Beweisen Sie:

- $O(6{,}94 \cdot \log_{13} n) = O(21{,}53 \cdot \log_2 n)$
- $O(1) \subset O(n)$
- $O(n) \subset O(n \log n)$
- $O(n \log n + \log n) \subset O(n \log n)$

n) Sie wollen einen Heap über Kundendaten aufbauen. Die Daten sollen vom Heap nach Vorname und nach Nachname priorisiert herausgegeben werden. Warum ist das ein Problem? Wie müsste man die Aufgabenstellung korrigieren?

Hinweis: Es mag helfen, die Begriffe *Halbordnung* und (*strenge*) *Totalordnung* aus der Mathematik nachzuschlagen (z. B. auf der Wikipedia).

Aufgabe 2: Praxis

a) Wandeln Sie Ihre Häschenkette aus den Vorübungen in eine Liste um, die Daten von einem zur Compilezeit wählbaren Typen vorhält. Ergänzen Sie, so noch nicht geschehen, Klassen für einen Iterator und für einen Listenknoten.

b) Schreiben Sie, sofern noch nicht geschehen:

- eine Funktion zum Einfügen in das Array der Vorübungen
- eine Funktion zum Löschen aus dem Array der Vorübungen
- eine Funktion zum Einfügen in die Liste der Vorübungen
- eine Funktion zum Löschen in der Liste der Vorübungen

c) Schreiben Sie eine Funktion `heapify`. Diese Funktion soll ein Array von Ihrem selbstgeschriebenen Arraytypen aus den Vorgängerübungen annehmen und auf diesem Array eine Heapstruktur für einen Max-Heap einrichten.

Es mag helfen, für die `heapify`-Funktion eine Hilfsfunktion `sift_up` zu erstellen. Es könnte ebenfalls hilfreich sein, eine Klasse heap als Wrapper für das Array aus den Vorübungen zu programmieren (also mit eigener, array-unabhängiger Größe).

Testen Sie Ihre Funktion `heapify` mit einer geeigneten `main`. Erstellen Sie zufällige Heaps und prüfen Sie mit einer weiteren Hilfsfunktion, `check_heap`, ob jeder Eintrag in inneren Knoten in den Ergebnisheaps die Heap-Eigenschaft erfüllt.

d) Schreiben Sie eine Funktion `extract_max`. Diese Funktion soll als Parameter ein als Heap aufgebautes Array annehmen und aus dem Array die Wurzel entfernen, wobei die Heapstruktur erhalten bleibt. Halten Sie die aktuelle Größe des Heaps in einer gesonderten Variablen (d. h. machen Sie bei `extract_max` nicht das Array kleiner, sondern nur den Heap).

Testen Sie Ihren Code in einer geeigneten `main`-Funktion.

e) Schreiben Sie eine Funktion `heap_sort`, welche ein gegebenes Array mit dem Verfahren aus der Vorlesung sortiert.

Testen Sie Ihren Code in einer geeigneten `main`-Funktion.

f) Schreiben Sie eine Funktion `binary_search`, welche auf einem sortierten Array mit dem Verfahren aus der Vorlesung prüft, ob ein bestimmter Wert enthalten ist.

g) Schreiben Sie eine Klasse `fifo_queue`. Diese soll die Funktionalität einer Queue zur Verfügung stellen, ohne dabei eine eigene Container-Implementierung zu enthalten. Nutzen Sie stattdessen Ihre eigene Listenklasse aus den Vorübungen.

h) Messen Sie, wie lange auf Ihrem Rechner das Suchen...

- ... in einem unsortierten Array,
- ... in einer unsortierten Liste und
- ... in einem unsortierten Heap

braucht. Achten Sie darauf, die Datenstrukturen hinreichend (aber gleich) groß und mit zufälligem (aber gleichem) Inhalt zu wählen. Können Sie die Laufzeitergebnisse aus der Vorlesung bestätigen?